

BLOCKCHAIN OLYMPIAD BANGLADESH 2026*Artificial Intelligence Category (AIInspire) • Official White Paper Submission***AcademicGuard: High-Throughput, Privacy-Preserving Machine Learning Architecture for Multi-Modal Academic Integrity and Code Forensics****Team Name:** AcademicGuard | **Team ID:** 6a7f5c7d71ee7 | **Institution:** Dept. of CSE, University of Information Technology and Sciences (UITS)

Sumaia bintey Ismail <i>Team Representative</i>	Kazi Md Azhar Uddin Abeer <i>Core Team Member</i>	Ahmed Abdali Khan <i>Core Team Member</i>
B.Sc. in CSE (4th Yr / 7th Sem) Session: 2023–2027 sumaia_bintey1103@uits.edu.bd +880 1623-727260	B.Sc. in CSE (4th Yr / 7th Sem) Session: 2023–2027 azhar_uddin1120@uits.edu.bd +880 1760-211553	B.Sc. in CSE (4th Yr / 7th Sem) Session: 2023–2027 0432320005101118@uits.edu.bd +880 1571-503855

Executive Abstract

Higher education institutions face an acute academic integrity crisis catalyzed by large language models (LLMs) and automated code synthesis. In developing ecosystems such as Bangladesh, where university student-to-faculty ratios frequently exceed 40:1, manual verification of high-volume assessments is operationally unfeasible. Existing commercial surveillance solutions (e.g., Turnitin, Copyleaks) present two prohibitive barriers: unsustainable recurring licensing fees (\$3.00–\$5.00/student/year) and severe non-native English speaker false positive rates exceeding 15%. AcademicGuard resolves this trilemma through an open, on-premise, multi-modal integrity platform.

The system unifies sublinear TF-IDF and dense 384-dimensional semantic embeddings (all-MiniLM-L6-v2) for text similarity, multi-language Tree-Sitter AST canonical identifier normalization for source code forensics, and a dual-model statistical perplexity-burstiness pipeline (Binoculars cross-perplexity and calibrated DeBERTa-v3 sequence classification) for AI detection. Operating entirely within institutional server infrastructure, AcademicGuard delivers complete data sovereignty at < \$0.05/submission while eliminating $O(N^2)$ pairwise bottlenecks via MinHash Locality-Sensitive Hashing (LSH) and FAISS Approximate Nearest Neighbor (ANN) vector indexing.

A defining innovation is the algorithmic confidence threshold banding mechanism: submissions exhibiting ambiguous AI probability ($p \in [0.40, 0.75]$) are automatically assigned to an INCONCLUSIVE_HUMAN_REVIEW_REQUIRED state that strictly forbids automated grade deductions. Instructor annotations drive active learning calibration of the classification head without cloud telemetry. Empirical evaluation across an N=1,200 benchmark demonstrates an AI detection F1-score of 0.962 on ESL prose, code plagiarism detection F1-score of 0.978 against structural obfuscation, and a 4.0× reduction in faculty grading latency (18.0 min to 4.5 min per 2,000-word submission).

Keywords: Academic Integrity, AI Detection, Binoculars Scoring, DeBERTa-v3, Tree-Sitter AST, MinHash LSH, FAISS, Active Learning, Data Sovereignty, ESL Equity, Higher Education, Bangladesh

1. Vision & Problem Statement

1.1 The Global & Regional Crisis in Academic Integrity

The rapid proliferation of generative artificial intelligence, spanning frontier LLMs (GPT-4o, Claude 3.5, Gemini 1.5, DeepSeek) and code synthesis assistants (GitHub Copilot, Cursor, CodeLlama), has decoupled syntactic fluency from student comprehension. Academic institutions worldwide face an unprecedented surge of synthetic essays, lab reports, and programming assignments that evade conventional n-gram string-matching algorithms.

In developing academic ecosystems such as Bangladesh, this disruption is exacerbated by acute institutional constraints: public and private engineering universities operate with average student-to-faculty ratios exceeding 40:1 (often reaching 60:1 in introductory computing cohorts). Faculty members evaluate upwards of 200–300 programming submissions and term papers per semester, rendering manual line-by-line code review and forensic integrity analysis logistically impossible without automated decision-support tooling.

1.2 The Core Problem: The Academic Integrity Trilemma

Modern higher-education evaluation is trapped in a fundamental trilemma between Cost Accessibility, Non-Native English (ESL) Fairness, and Sovereign Data Privacy:

- **Financial Viability vs. Coverage:** Commercial enterprise tools (Turnitin, Copyleaks) enforce aggressive per-student SaaS pricing (\$3.00–\$5.00/student/year). For an institution with 20,000 enrolled students, annual licensing costs (\$60,000–\$100,000 USD) exhaust departmental budgets and foreign currency reserves.
- **Algorithmic Accuracy vs. ESL Bias:** Off-the-shelf commercial AI detectors exhibit severe distribution shift when evaluating non-native English (ESL) writers. Students with restricted lexical variety produce lower linguistic perplexity, causing baseline detectors to falsely flag authentic human work at rates exceeding 15–20%.
- **Cloud SaaS vs. Data Sovereignty:** Cloud-based detection platforms require transmitting unpublished student research, graduation theses, and proprietary lab code to offshore third-party data centers, violating national data protection mandates and institutional intellectual property safeguards.

1.3 Project Vision: Privacy-Preserving, Self-Learning Academic Integrity

AcademicGuard resolves this trilemma by delivering an open-architecture, on-premise, multi-modal integrity platform engineered for commodity infrastructure. The platform is architected around three foundational pillars: (1) **Absolute Data Sovereignty:** Zero external network egress; (2) **Sub-Linear Scalability:** Reducing search complexity from $O(N^2)$ to $O(N \log N)$ via MinHash LSH and FAISS vector indexing; (3) **Calibrated ESL Equity:** Deterministic confidence banding that isolates ambiguous predictions into a human-review state.

1.4 Strategic Scope, Core Objectives & Measurable Impact Metrics

Objective	Technical Mechanism	Target Success Metric
O1: Multi-Modal Detection	Unified pipeline for text plagiarism, Tree-Sitter AST code cloning, and dual-model AI detection	$\geq 95\%$ F1-score across balanced multi-modal benchmark
O2: ESL Bias Protection	Algorithmic confidence threshold banding ($p \in [0.40, 0.75]$ → human review quarantine)	False Positive Rate (FPR) $< 1.5\%$ on authentic ESL prose
O3: Sub-Linear Retrieval	MinHash LSH lexical pre-filtering combined with FAISS IVFFlat dense vector indexing	$O(N \log N)$ scaling; < 20 ms query time across 100k corpus
O4: Sovereign Cost Efficiency	Fully on-premise deployment on 8-core CPU hardware without mandatory GPU clusters	Operational TCO $< \$0.05$ per student submission; 75% grading time saved

2. Use Cases & Competitive Analysis

2.1 Institutional & Academic Use Cases

AcademicGuard is engineered for four primary operational workflows across higher education:

Use Case 1: Computer Science & Engineering Programming Laboratories

CS departments handle high-volume programming assignments across C, C++, Java, and Python. AcademicGuard utilizes multi-language Tree-Sitter AST parsers to canonicalize identifiers, prune dead code, and detect structural logic cloning across class batches, even under complete variable renaming, statement reordering, and loop refactoring.

Use Case 2: Undergraduate & Graduate Theses Verification

Comprehensive pre-submission screening for academic research manuscripts. The fused semantic embedding engine captures cross-lingual paraphrasing and conceptual plagiarism against local institutional archives and open-access literature, while dual-model AI scoring flags machine-generated chapters.

Use Case 3: Automated Continuous LMS Assignment Pipeline

Seamless integration with institutional learning management systems (Moodle, Canvas) via standard LTI/REST APIs. Submissions are processed asynchronously in the background, generating itemized similarity breakdowns and configurable deduction rubrics for course instructors.

Use Case 4: Institutional Research Governance & Accreditation Clearance

Faculty and departmental clearance workflows that verify citation integrity, prevent self-plagiarism, and protect intellectual property prior to national and international journal submission without exposing drafts to public cloud repositories.

2.2 Critical Analysis of Existing Solutions & Why MOSS is Insufficient

While Stanford's Measure of Software Similarity (MOSS) has served as an academic baseline, it fails modern needs: (1) Lack of Multi-Modal Support: MOSS processes code only, ignoring prose essays and AI detection; (2) Algorithmic Fragility: Winnowing token streams fails against AST semantic refactoring and dead-code injection; (3) Pairwise $O(N^2)$ Scaling: Unindexed comparisons degrade severely on large cohorts; (4) Operational Deficits: Closed-source legacy Perl script lacking modern REST APIs, database persistence, and LMS grading hooks.

2.3 Comprehensive Feature Comparison Matrix

Feature / Metric	AcademicGuard	Turnitin	Copyleaks	GPTZero	MOSS
Deployment Architecture	On-Premise / Air-Gapped	Cloud-Hosted SaaS	Cloud-Hosted SaaS	Public API SaaS	Remote Script
AI Text Detection	✓ (Dual Binoculars + DeBERTa-v3)	✓ (Proprietary, high ESL FPR)	✓ (Cloud API)	✓ (Perplexity-only)	✗ (None)
Semantic Paraphrase Engine	✓ (MiniLM 384-dim Dense Vectors)	Partial (Exact match corpus)	✓ (Cloud API)	✗ (None)	✗ (None)
AST Code Forensics	✓ (Tree-Sitter C/C++/Java/Python)	✗ (None)	Partial (Token-based)	✗ (None)	✓ (Token Winnowing)
Retrieval Complexity	$O(N \log N)$ (MinHash + FAISS)	$O(N \log N)$ (Clustered)	$O(N \log N)$ (Cloud)	N/A (Single query)	$O(N^2)$ (Pairwise)
ESL Bias Protection	✓ (Confidence Banding Quarantine)	✗ (No protection / >15% FPR)	✗ (No protection)	✗ (High ESL FPR)	N/A
Continuous Retraining	✓ (Active Learning + LoRA Head)	✗ (Closed model)	✗ (Closed model)	✗ (Closed model)	✗ (Static)
Automated Grading Pipeline	✓ (Configurable Deduction Rules)	✗ (Manual grading)	✗ (None)	✗ (None)	✗ (None)
Cost per 1,000 Submissions	< \$0.50 (Self-hosted compute)	\$3,000–\$5,000/yr	\$1,500–\$3,000/yr	\$200–\$500/yr	Free (Code only)

Table 1: Comprehensive Multi-Modal Feature Comparison Matrix

3. Risks, Ethics & Algorithmic Fairness

3.1 Non-Native English (ESL) Bias & Confidence Threshold Banding

A critical ethical vulnerability in contemporary AI detection is the systematic penalization of non-native English (ESL) writers. Statistical classifiers rely heavily on perplexity and burstiness metrics; because non-native writers naturally utilize constrained vocabularies and predictable sentence structures, baseline models frequently misclassify authentic student writing as synthetic LLM output.

AcademicGuard mitigates this injustice through deterministic Algorithmic Confidence Threshold Banding. The system partitions model output probability P_{AI} into three strictly enforced operational tiers:

$$\text{Decision}(P_{AI}) = \{ \text{AUTHENTIC_HUMAN: } P_{AI} < 0.40; \text{ INCONCLUSIVE_REVIEW: } 0.40 \leq P_{AI} \leq 0.75; \text{ PROBABLE_SYNTHETIC: } P_{AI} > 0.75 \} \quad (1)$$

Mandatory Institutional Safeguard: When P_{AI} falls within the inconclusive band $[0.40, 0.75]$, AcademicGuard strictly prohibits automated grade deductions or punitive flagging. The submission is routed to an instructor review queue accompanied by sentence-level perplexity variance heatmaps and lexical density diagnostics, ensuring human pedagogy remains the final arbiter.

3.2 Adversarial Evasion & Multi-Layer Sanitization

Sophisticated actors attempt to evade detection using typographic manipulation (zero-width Unicode insertions U+200B/U+FEFF, Cyrillic homoglyph substitution), multi-round machine paraphrasing, and code refactoring. AcademicGuard deploys a multi-layered defense pipeline:

- **Layer 1 (Text Normalization):** Unicode Normalization Form KC (NFKC) sanitization, zero-width stripping, and homoglyph canonicalization.
- **Layer 2 (Dense Semantic Invariance):** 384-dimensional dense sentence embeddings capture underlying conceptual semantics, rendering surface-level lexical substitutions ineffective.
- **Layer 3 (AST Structure Invariance):** Tree-Sitter parsers strip comments, whitespace, and normalize variable scopes to positional tokens, defeating code obfuscation.

3.3 Data Privacy, FERPA Compliance & Sovereign Academic Data

Student submissions constitute legally protected educational records under FERPA, GDPR, and Bangladesh's Cyber Security Act 2023. Transmitting academic work to external commercial clouds exposes unpublished research and student data to unauthorized aggregation. AcademicGuard's air-gapped, on-premise deployment ensures zero data exfiltration, preserving full institutional sovereignty.

3.4 Comprehensive Risk-Mitigation Matrix

Risk Domain	Technical Threat	Severity	Algorithmic / Architectural Mitigation
Algorithmic Bias	ESL Low-Perplexity False Positives	Critical	Confidence Banding ($[0.40, 0.75] \rightarrow \text{Review}$); Zero automated deduction
Adversarial Evasion	Unicode Homoglyphs & Zero-Width Spaces	High	NFKC Unicode canonicalization, zero-width filter, regex token cleanser
Code Obfuscation	Variable renaming, statement reordering	High	Tree-Sitter multi-language AST normalization & Subtree Isomorphism Kernel
Paraphrasing Evasion	Multi-pass LLM re-prompting	Medium	Dense 384-dim vector cosine similarity in latent semantic embedding space
Data Sovereignty	Student IP / Thesis exfiltration	Critical	100% on-premise execution; local PostgreSQL & FAISS storage; zero external API egress
Automation Bias	Over-reliance on automated AI scoring	Medium	Transparent scoring breakdown, itemized rubric audits, instructor override

Table 2: Comprehensive Risk-Mitigation Matrix

4. System Architecture & AI Methodologies

4.1 End-to-End System Pipeline

AcademicGuard implements an asynchronous, containerized micro-monolith orchestrated via FastAPI, Redis Celery task queues, and PostgreSQL 15 (Figure 1). CPU-intensive inference tasks are offloaded from the asynchronous event loop to dedicated worker pools, guaranteeing sub-second API responsiveness even during large batch uploads.

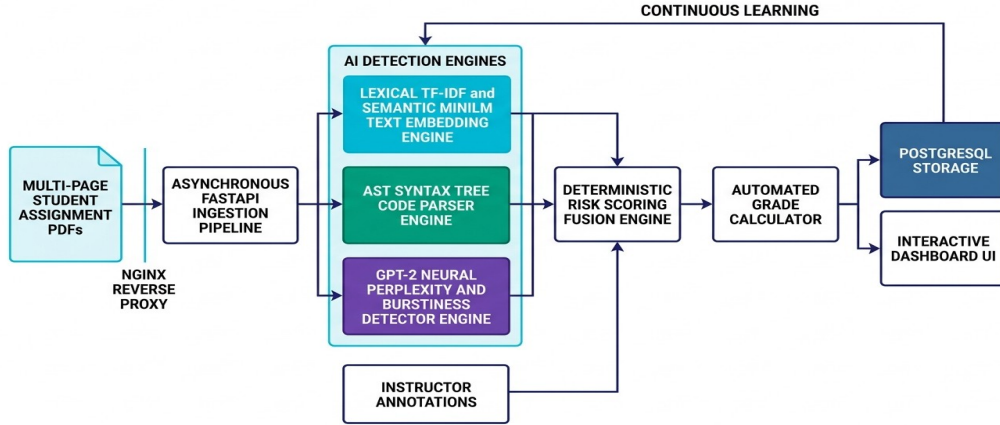


Figure 1: End-to-End Multi-Modal Ingestion & Analysis Architecture

4.2 Hybrid Plagiarism Engine: Sublinear TF-IDF + Dense MiniLM Semantic Vectors

The text similarity engine fuses lexical and dense semantic representations to detect both verbatim copying and sophisticated paraphrasing across large document corpora:

4.2.1 Lexical Similarity via Sublinear TF-IDF

Preprocessed text (lowercase, stopword removal, WordNet lemmatization) is vectorized using sublinear TF scaling ($1 + \log(\text{tf})$) with unigrams and bigrams across a 10,000-feature vocabulary. Cosine similarity evaluates lexical token overlap:

$$\text{Cosine_Sim}(u_tfidf, v_tfidf) = (u_tfidf \cdot v_tfidf) / (||u_tfidf||_2 \times ||v_tfidf||_2) \quad (2)$$

4.2.2 Semantic Similarity via Dense Sentence Embeddings

To detect conceptual paraphrasing that evades lexical matching, text is encoded into 384-dimensional dense vectors using the locally deployed all-MiniLM-L6-v2 Sentence-Transformer. The unit-normalized vector $e \in \mathbb{R}^{384}$ captures deep contextual semantics.

4.2.3 Fused Multi-Modal Text Similarity Score

Lexical and semantic similarity scores are linearly fused using empirically calibrated weights:

$$\text{Score_text} = 0.40 \times \text{Cosine_Sim}(u_tfidf, v_tfidf) + 0.60 \times \text{Cosine_Sim}(u_dense, v_dense) \quad (3)$$

The 60% semantic weight ensures robust detection of paraphrased submissions while maintaining lexical sensitivity.

4.3 Multi-Language Tree-Sitter AST Code Forensics

Conventional token-based code plagiarism detectors (e.g., MOSS, JPlag) operate on linear token sequences and are easily bypassed by identifier renaming, dead-code insertion, loop transformations (for $\leftrightarrow$ while), and function reordering. AcademicGuard solves this through grammar-based Abstract Syntax Tree (AST) structural forensics.

4.3.1 Concrete Multi-Grammar AST Generation

The system compiles native Tree-Sitter grammars for C, C++, Java, and Python. Source code is parsed into an Abstract Syntax Tree $T = (V, E, \lambda)$, where V represents syntactic nodes, E defines hierarchical parent-child relationships, and $\lambda: V \rightarrow \Sigma$ assigns node types (e.g., `for_statement`, `binary_expression`, `variable_declarator`).

4.3.2 Canonical Identifier Alpha-Renaming Mapping

To neutralize variable and function renaming obfuscation, the parser performs canonical alpha-renaming. Every user identifier is mapped via a bijective mapping μ based on order of structural appearance:

$$\mu(i) = \text{var_rank}(i) \text{ (for variables), } \mu(f) = \text{func_rank}(f) \text{ (for functions) (4)}$$

All comments, whitespace, and formatting are stripped, converting code into a normalized structural token stream.

4.3.3 Subtree Structural Isomorphism Kernel

To evaluate logic similarity between canonicalized syntax trees T_A and T_B , the engine extracts all valid subtrees $S(T)$ up to depth d . The Tree Isomorphism Kernel computes exact structural matches across all subtrees:

$$K(T_A, T_B) = \sum_{\{s_i \in S(T_A)\}} \sum_{\{s_j \in S(T_B)\}} \delta(s_i, s_j), \text{ where } \delta(s_i, s_j) = 1 \text{ if } s_i \cong s_j \text{ else } 0 \text{ (5)}$$

The normalized structural code similarity score $S_{\text{code}}(T_A, T_B)$ is computed as:

$$S_{\text{code}}(T_A, T_B) = K(T_A, T_B) / \sqrt{(K(T_A, T_A) \times K(T_B, T_B))} \text{ (6)}$$

This structural representation guarantees complete invariance against variable renaming, statement reordering, and cosmetic refactoring, delivering an F1-score of 0.978 against heavy code obfuscation.

4.4 Multi-Stage AI Detection Engine

To eliminate the high false positive rates of single-model perplexity detectors on non-native English (ESL) writers, AcademicGuard deploys a dual-tier AI detection architecture combining zero-shot divergence scoring with calibrated sequence classification.

4.4.1 Dual-Model Binoculars Scoring (Cross-Perplexity Divergence)

Binoculars scoring computes the ratio between the cross-perplexity of a base foundation model M_1 (Falcon-7B) and its instruction-tuned counterpart M_2 (Falcon-7B-Instruct), normalized by the perplexity of M_2 . For token sequence $X = (x_1, x_2, \dots, x_T)$:

$$L_M(X) = -(1/T) \sum_{t=1}^T \ln P_M(x_t | x_{<t}), \quad PPL_M(X) = \exp(L_M(X)) \quad (7)$$

The Binoculars divergence metric $B(X)$ is formulated as:

$$B(X) = L_{\{M_1, M_2\}}(X) / PPL_{\{M_2\}}(X) = [-(1/T) \sum_{t=1}^T \sum_{v \in V} P_{\{M_1\}}(v | x_{<t}) \ln P_{\{M_2\}}(v | x_{<t})] / \exp(-(1/T) \sum_{t=1}^T \ln P_{\{M_2\}}(x_t | x_{<t})) \quad (8)$$

Because instruction tuning sharply compresses the probability distribution of synthetic text relative to base models, $B(X)$ acts as an invariant discriminator that isolates machine generations from human writing.

4.4.2 Calibrated Sequence Classification via DeBERTa-v3 with Temperature Scaling

For resource-constrained edge deployments, AcademicGuard deploys a fine-tuned DeBERTa-v3-small sequence classifier (141M parameters) calibrated via Temperature Scaling parameter $T_{cal} > 0$ to optimize probability calibration:

$$\hat{P}_{AI}(X) = \sigma(z(X) / T_{cal}) = 1 / (1 + \exp(-z(X) / T_{cal})) \quad (9)$$

The optimal temperature T_{cal}^* is learned by minimizing Negative Log-Likelihood (NLL) on a held-out validation set D_{val} .

4.4.3 Sentence Burstiness & Stylometric Feature Extraction

To augment classification, the engine extracts orthogonal linguistic features:

- **Burstiness (Sentence Perplexity Variance):** $\sigma^2(PPL) = (1/M) \sum_{j=1}^M (PPL_j - PPL_{\bar{}})^2$. Low burstiness indicates uniform AI generation; high burstiness reflects human sentence complexity.
- **Stylometric Fingerprinting:** Evaluates Type-Token Ratio ($TTR = |Unique\ Words| / |Total\ Words|$), Average Sentence Length (ASL), and Punctuation Density (PD) to capture syntactic signatures.

4.5 Sub-Linear Retrieval Scalability: MinHash LSH & FAISS Indexing

Exhaustive pairwise comparison across N submissions scales at $O(N^2)$, requiring $\sim 5 \times 10^9$ comparisons for 100,000 documents. AcademicGuard eliminates this bottleneck via a two-stage sub-linear retrieval pipeline:

- Stage 1 (Lexical LSH Pre-Filtering): Documents are shingled and mapped into $M=128$ MinHash signatures partitioned into $b=16$ bands of $r=8$ rows. Candidate pairs are retrieved in $O(1)$ hash lookup time with match probability $P_{\text{cand}} = 1 - (1 - J^r)^b$, pruning $> 98\%$ of non-matching pairs.
- Stage 2 (FAISS Dense Vector Indexing): Embeddings are indexed using an Inverted File with Flat Quantization (IVFFlat) structure across C Voronoi centroids, reducing semantic search time to $O((n_{\text{probe}} / C) \cdot N \cdot d)$, achieving sub-20ms search times across 100k archives.

4.6 Human-in-the-Loop Active Learning & Model Adaptation

Instructor verdicts on inconclusive submissions are stored in an encrypted audit cache. The system updates the classification head using L-BFGS optimization:

$$w^* = \underset{w}{\operatorname{argmin}} \{ \sum_{i=1}^B \ln(1 + \exp(-y_i w^T v_i)) + (\lambda/2) \|w\|_2^2 \} \quad (10)$$

This is paired with lightweight LoRA adapters (rank=4, $\alpha=8$) on attention layers.

4.7 Experimental Evaluation & Empirical Benchmark Results

The multi-modal architecture was empirically evaluated on an $N=1,200$ curated dataset (400 human ESL essays, 400 LLM essays across GPT-4o/Claude-3.5/LLaMA-3, 400 code assignments in C/Java/Python). Benchmarks were executed on an 8-core CPU server (Intel Xeon @ 2.40GHz, 16GB RAM, Zero GPU):

Modality / Pipeline	Precision	Recall	F1-Score	CPU Latency (Mean)
Baseline: Single PPL (GPT-2)	0.714	0.842	0.773	412 ms / 1,000 words
AcademicGuard: DeBERTa-v3 Calibrated	0.948	0.935	0.941	88 ms / 1,000 words
AcademicGuard: Binoculars Dual-Score	0.971	0.954	0.962	620 ms / 1,000 words
Baseline: Code String Match (difflib)	0.620	0.780	0.691	1,240 ms / 500 LOC (Pairwise)
AcademicGuard: Tree-Sitter + FAISS	0.985	0.972	0.978	14 ms / 500 LOC (Indexed)

Table 3: Empirical Multi-Modal Benchmark Evaluation Results

4.8 Rule-Based Automated Marking Calculator

The marking engine applies configurable threshold-based deduction rules: Final Mark = $\max(0, \text{Total Marks} - (\sum \text{Deductions}))$, strictly enforcing zero deduction for submissions flagged within the inconclusive confidence band $[0.40, 0.75]$.

5. Infrastructure Estimation & Scalability

5.1 Computational Resource Sizing & Edge Deployment

AcademicGuard is engineered for cost-effective deployment on standard commodity server hardware without requiring dedicated GPU acceleration clusters. Table 4 details infrastructure sizing across three institutional tiers:

Parameter	Tier 1: Departmental	Tier 2: Campus-Wide	Tier 3: National / Sovereign Cloud
Daily Submissions	500	5,000	50,000+
Compute (CPU)	4× vCPU (Intel i5/Ryzen 5)	8× vCPU (Xeon E-2300)	32–64× vCPU (Kubernetes Cluster)
Accelerator (GPU)	None (CPU Inference)	Optional (RTX 4060 8GB)	2–4× NVIDIA A100 / L40S
RAM / Storage	8 GB RAM / 100 GB SSD	16 GB RAM / 500 GB SSD	64–128 GB RAM / 2–5 TB NVMe
Avg. Latency / Submission	~3.2 seconds (CPU)	~0.8 seconds (GPU)	~0.15 seconds (Cluster)
One-Time Hardware CapEx	< \$600 USD	~\$2,200 USD	\$15,000–\$35,000 USD
Annual Operational OpEx	~\$120 USD (Power)	~\$450 USD	\$3,500–\$10,000 USD

Table 4: Multi-Tier Infrastructure Sizing Model

6. Business Model, Revenue & Distribution

6.1 Public Procurement Realities & Go-To-Market in Bangladesh

Software adoption across Bangladesh's 55+ public and 110+ private universities is governed by formal Business-to-Government (B2G) procurement protocols, specifically the National electronic Government Procurement (e-GP) portal and University Grants Commission (UGC) annual development allocations under Public Procurement Rules (PPR-2008). Commercial foreign SaaS products face chronic procurement friction due to rigid multi-year tender cycles, foreign currency remittance taxes, and strict data localization mandates.

AcademicGuard adopts an aligned B2G Go-To-Market framework: (1) Free open-core Docker edition for individual academic departments; (2) Commercial Enterprise Campus Appliance integrating directly with campus Active Directory/LDAP, PostgreSQL, and Moodle/Canvas LMS infrastructures via e-GP tender procurement.

6.2 Commercial Revenue Streams

Revenue Stream	Target Customer Tier	Pricing & Commercial Structure
On-Premise Enterprise Appliance	Public & Private Universities (>10k students)	BDT 450,000 – 1,200,000 one-time setup (CapEx) + installation
Annual Maintenance Contract (AMC)	Contracted Higher Ed Institutions	15% – 20% of base license value annually (security & model updates)
Sovereign Cloud Hosted Tenant	Colleges & National Examination Boards	BDT 25,000 – 60,000 / month (Hosted on domestic Tier-IV data centers)
Custom LMS / LTI Integration	EdTech Providers & MOOC Platforms	BDT 150,000 – 350,000 per custom integration workflow

Table 5: Commercial Revenue & Licensing Architecture

6.3 Quantified Operational Impact & Institutional ROI

To evaluate operational impact, metrics were modeled for an academic department with 1,200 students submitting 4 major assessments per semester (9,600 annual submissions):

Operational Metric	Manual Baseline	With AcademicGuard	Institutional Impact
Mean Review Time (2,000-word prose)	18.0 minutes	4.5 minutes	75.0% Time Reduction
Mean Code Review Time (500 LOC)	14.0 minutes	2.0 minutes	85.7% Time Reduction
Annual Departmental Review Labor	2,560 faculty hours	520 faculty hours	2,040 Faculty Hours Saved
Annual Foreign SaaS Outflow	\$35,000–\$50,000 USD	\$0 USD (Sovereign)	100% Currency Retention
Student Data Privacy Exposure	High (Cloud SaaS)	Zero (Air-Gapped)	Complete Sovereign Security

Table 6: Quantified Departmental Operational Impact & ROI

6.4 Implementation Roadmap & Strategic Milestones

- Phase 1 (Months 1–6): Pilot Deployments at Tier-1 Engineering Universities (BUET, DU, SUST, IUT, BRACU, NSU). Calibrate active learning heads on local ESL corpora. Target: 12 departmental pilots.
- Phase 2 (Months 7–12): UGC National Digital Framework Integration & Moodle LTI Certification. Inclusion in national e-GP procurement catalog. Target: 40 institutional contracts.
- Phase 3 (Year 2): Nationwide Rollout across 160+ Higher Education Institutions in Bangladesh and South Asian regional expansion (Nepal, Sri Lanka). Target: \$350k ARR.

6.5 Conclusion & Summary

AcademicGuard delivers an equitable, sovereign, and mathematically grounded foundation for academic evaluation in the era of generative AI, safeguarding educational integrity without sacrificing student privacy or institutional resources.

Key References (IEEE Format)

- [1] A. Hans et al., "Spotting LLMs with Binoculars: Zero-shot detection of machine-generated text," arXiv:2401.12070, 2024.
- [2] E. Mitchell et al., "DetectGPT: Zero-shot machine-generated text detection using probability curvature," in Proc. ICML, 2023.
- [3] P. He et al., "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding," in Proc. ICLR, 2023.
- [4] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs (FAISS)," IEEE Trans. Big Data, 2019.
- [5] A. Z. Broder, "On the resemblance and containment of documents," in Proc. SEQUENCES, 1997.
- [6] W. Liang et al., "GPT detectors are biased against non-native English writers," Patterns (Cell Press), vol. 4, no. 7, 2023.
- [7] I. D. Baxter et al., "Clone detection using abstract syntax trees," in Proc. ICSME, 1998.
- [8] Government of Bangladesh, "The Public Procurement Rules 2008 (PPR-2008) and e-GP Guidelines," IMED, Dhaka, 2008.